

Chapter 2:
SECURE SYSTEM DESIGN & SECURE DESIGN PRINCIPLES

By:Tagel W.(M.Sc.)

Outlines

- Secure Design Architecture
- Secure System Design
- Secure Design Principles
- Threat Modeling
- Secure Software Development Lifecycle

Secure Design Architecture

- is the high-level structure and organization of a software system, including components, modules, and interfaces.
- focuses on the physical and logical layout of a software system and how its components interact with each other.
- It also includes the selection of appropriate technologies and platforms to support the system.

Secure System Design

- The process of designing software systems to ensure security and resilient to attacks.
- Includes defining security requirements, identifying security risks, and selecting appropriate security controls to mitigate those risks.
- It focuses on the functional and non-functional requirements of a software system, as well as the overall system design.
- The goal of secure software design
  - ✓ to create systems that are resistant to attack and protect against unauthorized access, data breaches, and other security threats.

Secure System Design…

- Secure system design covers a wide range of disciplines:
  - Software engineering
  - Network design
  - Cryptography and
  - Risk management
- requires a deep understanding of both technologies used to build systems and threats that those systems face, as well as an awareness of the regulatory and compliance requirements that govern the use of the systems.

Secure Design Principles

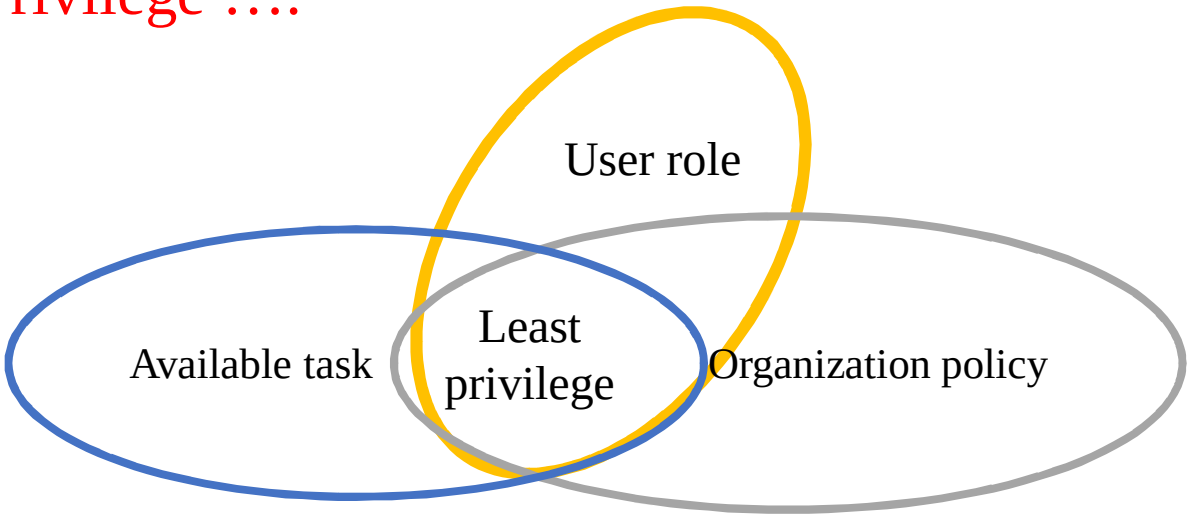
- is a set of guidelines and best practices used to build secure computer systems.
- These principles are designed to help developers and engineers create systems that are resistant to attack, protect against unauthorized access, and ensure the confidentiality, integrity, and availability of data.
- Some of the secure design principles are:
- Secure Communication: ensuring all communication channels between system components are secure and encrypted to prevent eavesdropping or tampering.

Secure Design Principles …

- Least Privilege: allocate the fewest privileges needed for a task, and for shortest duration necessary
- Every program and every user of the system should operate using the least set of privileges necessary to complete the job.
- It is related to military need-to-know principle access to sensitive information is granted only if essential to carrying out one's official duties.
- This limits damage that can result from an accident or error.

Secure Design Principles …

- Least Privilege ….



- E.g. a company's HR department might need access to the employee database to manage employee records, but they don't need access to the financial records. Similarly, the finance department might need access to financial records, but they don't need access to employee records.

Secure Design Principles …

- Economy of Mechanism

- A security mechanisms used in a system should be as simple and straightforward as possible.
- which means the fewer mechanisms that are used in a system, the less likely there are to be errors or vulnerabilities that can be exploited by attackers.
- It becomes easier to understand and verify their correctness.
- simple mechanisms tend to be easier to implement and maintain, which can make the system more reliable overall.

Secure Design Principles …

- Economy of Mechanism…

- The goal of minimizing the number of interfaces, simplifying design, minimizing external access to them, and restricting access to authorized parties is to improve the security and efficiency of a system.

- E.g. Use of automatic teller machines (ATMs) in banking. ATMs are designed with a simple and efficient mechanism that allows customers to withdraw cash, check their account balance, and perform other banking transactions.

- the mechanism used in ATMs is designed to be simple and efficient, minimizing the potential for errors and vulnerabilities.

  - The ATM software is designed to be secure and resilient to attacks with built-in security features(encryption, access controls, and authentication).

Secure Design Principles …

- Fail-Safe Defaults: a system or application should be designed to automatically deny access or restrict functionality in the event of an error or security breach.
- Ensures that security controls are enabled by default and that any exceptions are carefully considered.
- a system is configured to deny all network traffic by default, and then allows only specific traffic that is necessary for the system to function properly
- this principle is to minimize the potential damage caused by an error or security breach by limiting the amount of access or functionality that an attacker can gain.
- E.g1. A network router can use fail-safe defaults by defaulting to a closed port policy, ensuring that only authorized traffic is allowed through the router.

Secure Design Principles …

- Fail-Safe Defaults …

◆ E.g2. a user is automatically logs out by the system after a certain period of inactivity. Which prevents unauthorized access to the system in the event that a user forgets to log out or leaves computer unattended.

Or a system is automatically blocks access to a user account if there are too many failed login attempts.

Secure Design Principles …

- Complete Mediation

- The system should check the user's authorization to access the file every time when the user attempts to access it.

- It ensures that every access to a resource is properly authorized and that users are only granted access to the resources they are authorized to access.

- implemented using access control mechanisms such as role-based access control (RBAC) or attribute-based access control (ABAC)

- This prevents unauthorized access to the file, even if an attacker gains access to the user's account.

Secure Design Principles …

- Complete Mediation

- E.g. bank teller who needs to access a customer's account information would need to be authorized through the RBAC system every time they attempted to access the information. This ensures that the teller only has access to the specific customer accounts that they are authorized to access, and that they cannot access any accounts that they are not authorized to access.

- E.g. an e-commerce application that uses attribute-based access control to control access to customer data.

- In this scenario, the application would check the user's authorization to access customer data every time they attempted to access it, based on attributes such as user roles, job titles, location, time of day etc.

Secure Design Principles …

- Open Design

- Security mechanisms and protocols should be designed in an open and transparent manner, so that they can be reviewed and analyzed by a wide range of experts and stakeholders.

- This is to increase the likelihood that security vulnerabilities will be identified and addressed before they can be exploited by attackers.

- This principle reflects that security should not depend on the secrecy of the design or implementation.

Secure Design Principles …

- Open Design…

  - E.g. GnuPG(GPG) is an open-source encryption software that provides users with a way to encrypt and sign their emails, files, and other sensitive data.
  - GPG is based on the OpenPGP standard, which is a widely accepted standard for secure email communication.

- GPG uses a public key cryptography system, which means that users have two keys: a public key that can be shared with others, and a private key that should be kept secret.

To encrypt a message, the sender uses the recipient's public key to encrypt the message, and the recipient uses their private key to decrypt the message.

To sign a message, the sender uses their private key to add a digital signature to the message, which can be verified by the recipient using the sender's public key.

Secure Design Principles …

- Separation of Privilege

- Accessing to sensitive resources or functionality should be divided among multiple users or components, so that no single user or component has complete access or control.

- this principle is to limit the potential damage that can be caused by an attacker who gains access to a single user or component.

- E.g. A database server can use separation of concerns by separating database administration functions from user management functions, ensuring that a security breach in one area doesn't affect the security of the other area.

Secure Design Principles …

- Least Common Mechanism
- Keeping different parts of the system separate to minimize the impact of a security breach.
- The shared resources should be used as little as possible to minimize the potential damage that caused by an attacker who gains access to the shared resource or mechanism.
- Includes separating user accounts, data, and application components..
- E.g. In a multi-user system, each user should have their own separate home directory, rather than sharing a common directory.
- This ensures that if one user's account is compromised, the attacker cannot gain access to other users' files or data.

Secure Design Principles …

- Psychological Acceptability

- Ensures that security mechanisms and protocols are designed in a way that is acceptable and usable by users, without causing excessive stress, frustration, or confusion.

- designers and developers can ensure their products and services not only meet security standards but also meet the psychological needs and preferences of their users.

- This principle is to ensure that security measures are not perceived as barriers to productivity or usability, which can lead users to bypass or ignore them.

- E.g. If a password is rejected, the password changing program should state why it was rejected rather than giving a cryptic error message.

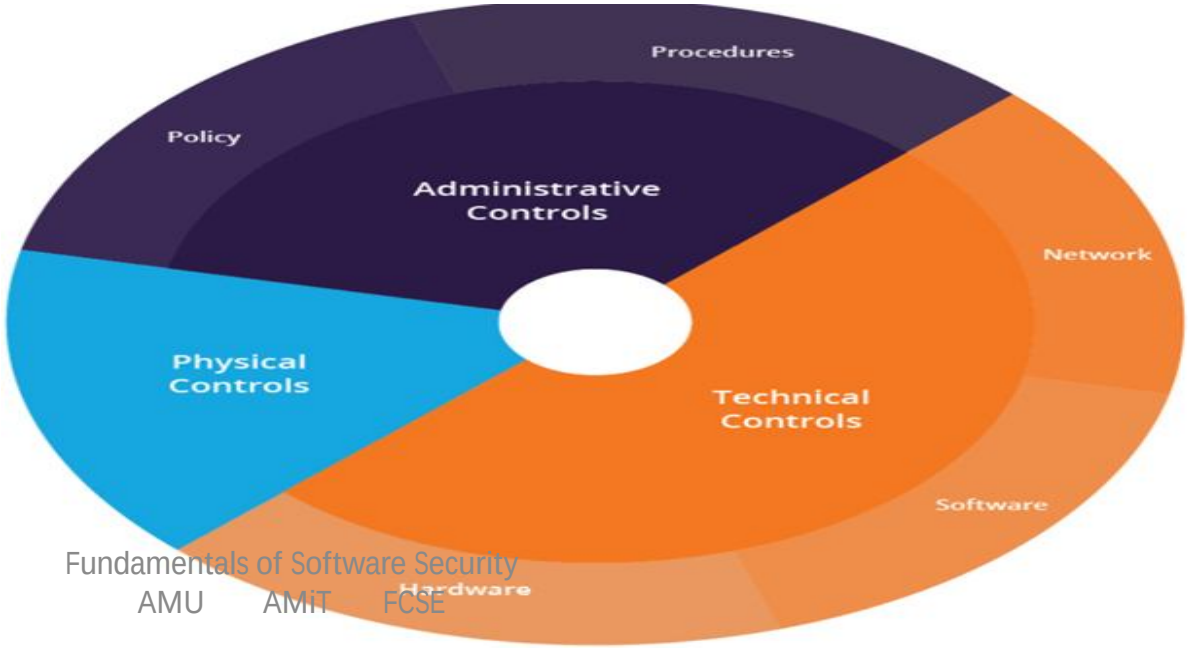
Secure Design Principles …

- Defense-in-Depth

- Provides multiple layers of security should be used to protect a system, rather than relying on a single security measure.

- This principle is to provide multiple barriers that attackers must overcome to compromise the system, which helps to reduce the likelihood of a successful attack.

- Defense in depth can be divided into three areas: Physical, Technical, and Administrative.



Secure Design Principles …

- Defense-in-Depth …
- Physical control: prevent physical access to systems(locked door, fences CCTV system etc.)
- Technical control: are hardware or software protect systems and resources(Firewalls, intrusion detection systems, encryption, and identification and authentication)
- Administrative control: consisting of policies or procedures directed at an organization's employees (e.g. training users, password management, incidence repones, access control policy etc.)

Threat Modeling

- It is a process identifies and evaluates potential security threats to a software system
- Used during design phase of software development to proactively identify potential vulnerabilities and determine how to mitigate them.
- The process of threat modeling involves following steps:
- Identify the system scope: boundaries system being modeled, including components, interfaces, and dependencies.
- Identify assets: assets the system needs to protect, e.g. data, user accounts, or system components.
- Identify potential threats: Identify potential attackers, motivations and vulnerabilities that could be exploited to compromise the system.

Threat modeling ….

- Evaluate threats: assess the likelihood and impact of each potential threat, and prioritize them based on the severity.
- Identify countermeasures: countermeasures to mitigate the identified threats such as access controls, encryption, or intrusion detection systems.
- Review and update: continuously review and update the threat model the system evolves in identifying and mitigating potential threats.

Secure Software Development Lifecycle

- is a set of practices and procedures designed to integrate security into every phase of the software development process.
- Goal secure SDLC: create software more resilient to security threats and vulnerabilities.
- Planning: security requirements and risk assessment.
- Requirements gathering: functional and non-functional requirements of the software are defined, security requirements.
- Design: architectural design of the software, security controls and mechanisms.
- Implementation: software is developed and tested. e.g. unit testing and code review for security vulnerabilities.
- Testing: software is tested for security vulnerabilities, including penetration testing and vulnerability scanning.
- Deployment: e.g. secure configuration management and hardening.
- Maintenance: patch management and incident response planning.

End of Chapter-2

Qn. if Any???

